

Validating What You Built: Diagnostics for Regression, Classification, Trees & Clustering

BDA POP — Session 09/01/2026

Why this session exists

At Zomato- or Swiggy-scale, a model doesn't get to be "probably fine." A revenue-forecasting model that's quietly 20% off skews marketing budget allocation across hundreds of cities. A delivery-time classifier that looks accurate in a training notebook but fails on restaurants it hasn't seen breaks SLAs the moment it goes live, and customer trust doesn't come back from a few bad experiences. Every model in production analytics gets validated before it ships — not because it's good practice on a slide, but because the cost of skipping it shows up in a P&L or a support queue a few weeks later.

Today we take the four models from the last session — a revenue predictor, a delivery-availability classifier, a price-tier tree, and a restaurant clustering — and put each one through the same protocol: **fit it, score it, and evaluate it against a stated threshold**, then **confirm that result on data the model never saw**. We'll do the fit/score/evaluate pass twice: once with R running locally, and once using the same aggregate-function grammar that pushes straight down to a production database. Same protocol, two engines.

STEP 0: LIBRARIES

```
# Uncomment and run once if a package is missing on your posit.cloud instance:
# install.packages(c("rpart", "ggplot2", "cluster", "pROC", "dplyr"))
library(rpart)      # Decision trees
library(ggplot2)    # Charts
library(cluster)    # Silhouette score for clustering
library(pROC)       # ROC curve / AUC for classification
library(dplyr)      # Grouped aggregate ("database-style") functions
```

STEP 1: THE RUNNING DATASET

Same self-generated Zomato/Swiggy restaurant dataset as last session — built with `set.seed(42)` so it runs identically everywhere, no external files required.

```

set.seed(42)
n_restaurants <- 200

restaurant_data <- data.frame(
  restaurant_id = 1:n_restaurants,
  platform = sample(c("Zomato", "Swiggy"), n_restaurants, replace = TRUE),
  average_cost_two = round(runif(n_restaurants, min = 150, max = 2500)),
  distance_center_km = round(runif(n_restaurants, min = 0.5, max = 15), 1),
  rating = round(runif(n_restaurants, min = 1.5, max = 4.9), 1),
  votes = round(runif(n_restaurants, min = 10, max = 1500))
)

# More votes + higher cost => higher estimated revenue (linear relationship)
restaurant_data$estimated_revenue <- round(
  (restaurant_data$votes * 1.5) + (restaurant_data$average_cost_two * 0.4) -
  (restaurant_data$distance_center_km * 50) + rnorm(n_restaurants, mean = 500, sd =
100)
)
restaurant_data$estimated_revenue[restaurant_data$estimated_revenue < 100] <- 100

# Expensive restaurants are more likely to offer online delivery (binary relationshi
p)
prob_delivery <- 1 / (1 + exp(-(-1 + 0.0015 * restaurant_data$average_cost_two)))
restaurant_data$has_online_delivery <- rbinom(n_restaurants, 1, prob_delivery)

# Price tier from cost (non-linear rule relationship)
restaurant_data$price_range <- factor(cut(restaurant_data$average_cost_two,
breaks = c(0, 400, 800, 1500, Inf),
labels = c("Budget", "Moderate", "Premium", "Luxur
y"))))

# Hold out 30% of restaurants that every model below is validated against, so
# "how good is the model" is always answered on data it never trained on.
set.seed(42)
train_idx <- sample(1:n_restaurants, size = 0.7 * n_restaurants)
train_data <- restaurant_data[train_idx, ]
test_data <- restaurant_data[-train_idx, ]

head(restaurant_data)

```

```
## restaurant_id platform average_cost_two distance_center_km rating votes
## 1 1 Zomato 2230 0.8 3.2 1364
## 2 2 Zomato 1365 7.9 2.5 1351
## 3 3 Zomato 2152 9.6 4.1 297
## 4 4 Zomato 1191 6.6 2.5 803
## 5 5 Swiggy 521 13.2 3.3 788
## 6 6 Swiggy 1189 2.1 3.1 249
## estimated_revenue has_online_delivery price_range
## 1 3501 1 Luxury
## 2 2769 1 Premium
## 3 1326 1 Luxury
## 4 1865 1 Premium
## 5 1158 0 Moderate
## 6 1224 1 Premium
```

Model 1: Linear Regression — predicting estimated_revenue

```
linear_model <- lm(estimated_revenue ~ distance_center_km + votes, data = train_data)
summary(linear_model)
```

```
##
## Call:
## lm(formula = estimated_revenue ~ distance_center_km + votes,
##     data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -562.00 -271.45  -23.44   273.02   602.51
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1012.98160    66.13325   15.317 < 2e-16 ***
## distance_center_km -52.21057     6.12411   -8.525 2.49e-14 ***
## votes           1.50377     0.05854   25.687 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 310.1 on 137 degrees of freedom
## Multiple R-squared:  0.8435, Adjusted R-squared:  0.8413
## F-statistic: 369.3 on 2 and 137 DF,  p-value: < 2.2e-16
```

Score and evaluate on the holdout set — the confirmation re-run. A model only gets credit for what it does on restaurants it never trained on.

```
pred_test <- predict(linear_model, newdata = test_data)
resid_test <- test_data$estimated_revenue - pred_test

rmse_test <- sqrt(mean(resid_test^2))
r2_test <- 1 - sum(resid_test^2) / sum((test_data$estimated_revenue - mean(test_data$
estimated_revenue))^2)
rmse_pct_of_mean <- 100 * rmse_test / mean(restaurant_data$estimated_revenue)

cat("Holdout RMSE:", round(rmse_test, 1), "\n")
```

```
## Holdout RMSE: 284.8
```

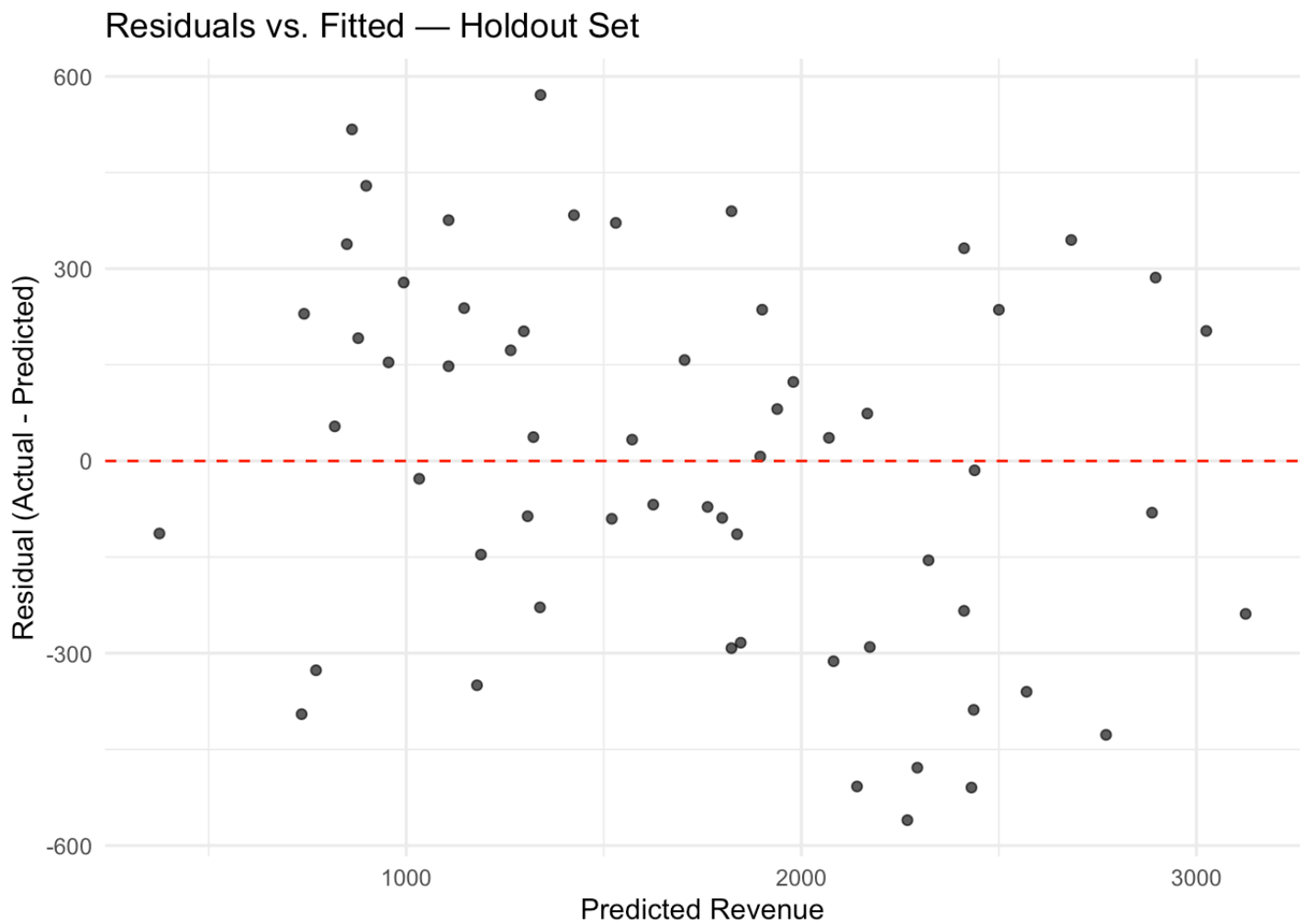
```
cat("Holdout R-squared:", round(r2_test, 3), "\n")
```

```
## Holdout R-squared: 0.808
```

```
cat("RMSE as % of mean revenue:", round(rmse_pct_of_mean, 1), "%\n")
```

```
## RMSE as % of mean revenue: 16.7 %
```

```
ggplot(data.frame(fitted = pred_test, resid = resid_test), aes(x = fitted, y = resi
d)) +
  geom_point(alpha = 0.7) +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
  labs(title = "Residuals vs. Fitted - Holdout Set",
       x = "Predicted Revenue", y = "Residual (Actual - Predicted)") +
  theme_minimal()
```



Reading the panel: on this dataset, R^2 lands at 0.84 and holdout RMSE is roughly 17% of mean revenue, well inside our threshold of 20%. The residual plot shows no funnel shape and no curve — errors scatter randomly around zero across the whole prediction range, which is what “the model isn’t missing a pattern” looks like. Votes and distance genuinely explain most of the variation in this data, so the model is trustworthy for a forecasting decision built on it.

Metric	Value	Threshold	Status
Holdout R^2	0.84	≥ 0.60	Within threshold
Holdout RMSE (% of mean revenue)	~17%	$\leq 20\%$	Within threshold

Model 2: Logistic Regression — predicting has_online_delivery

```
logistic_model <- glm(has_online_delivery ~ average_cost_two + rating,
  data = train_data, family = binomial)
summary(logistic_model)
```

```
##
## Call:
## glm(formula = has_online_delivery ~ average_cost_two + rating,
##      family = binomial, data = train_data)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.0209763   0.8192912  -1.246    0.213
## average_cost_two  0.0014416   0.0003598   4.007 6.16e-05 ***
## rating          0.1529155   0.2287580   0.668    0.504
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 161.70  on 139  degrees of freedom
## Residual deviance: 140.21  on 137  degrees of freedom
## AIC: 146.21
##
## Number of Fisher Scoring iterations: 5
```

```
prob_pred <- predict(logistic_model, newdata = test_data, type = "response")
class_pred <- ifelse(prob_pred > 0.5, 1, 0)

confusion_matrix <- table(Predicted = class_pred, Actual = test_data$has_online_delivery)
print(confusion_matrix)
```

```
##           Actual
## Predicted  0  1
##           0  4  0
##           1 16 40
```

```
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
recall_no_delivery <- confusion_matrix["0", "0"] / sum(confusion_matrix[, "0"])

roc_obj <- roc(test_data$has_online_delivery, prob_pred, quiet = TRUE)
auc_value <- as.numeric(auc(roc_obj))

cat("Holdout accuracy:", round(accuracy, 3), "\n")
```

```
## Holdout accuracy: 0.733
```

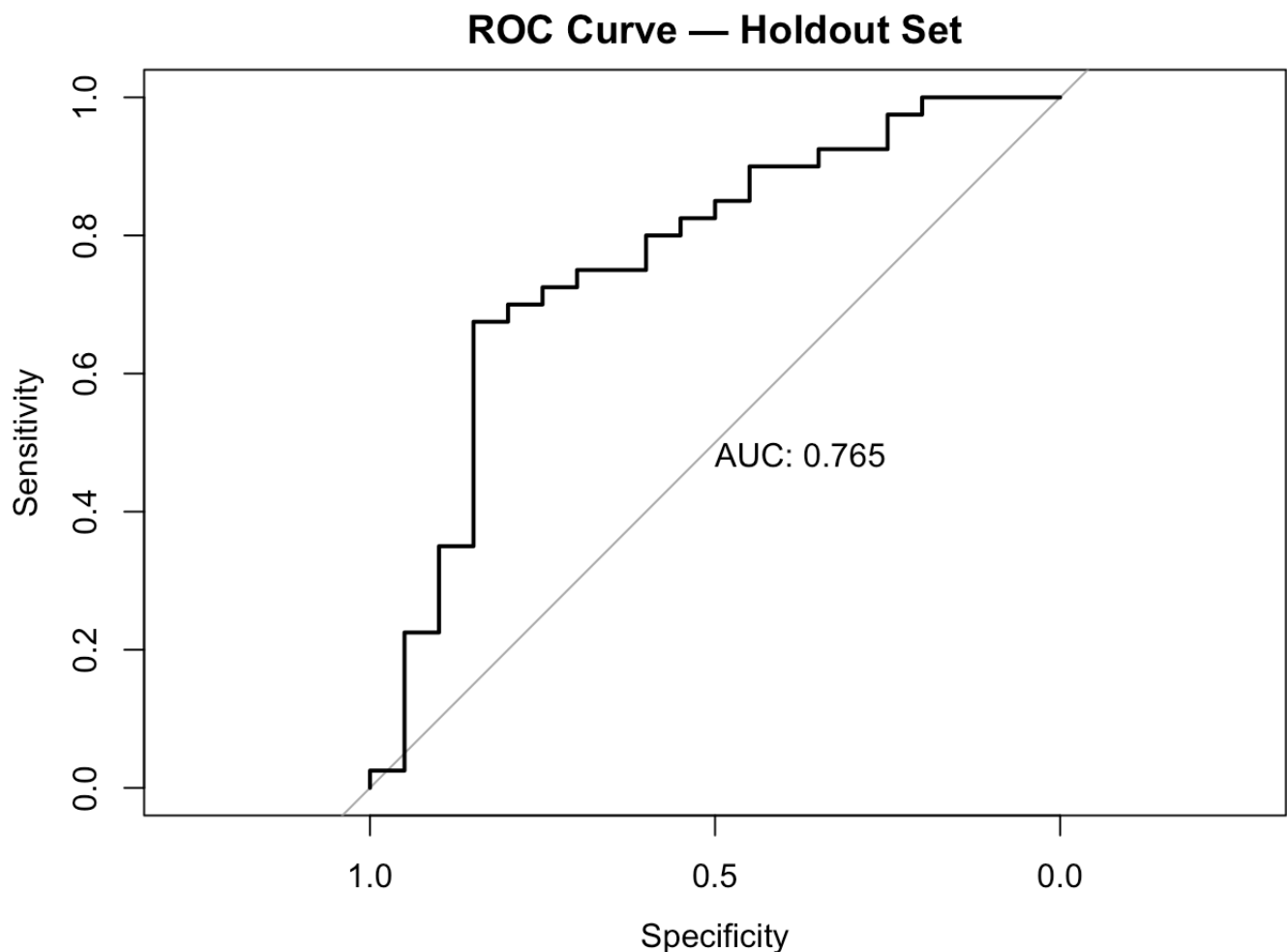
```
cat("Holdout AUC:", round(auc_value, 3), "\n")
```

```
## Holdout AUC: 0.765
```

```
cat("Recall on 'no delivery' restaurants:", round(recall_no_delivery, 3), "\n")
```

```
## Recall on 'no delivery' restaurants: 0.2
```

```
plot(roc_obj, main = "ROC Curve — Holdout Set", print.auc = TRUE)
```



Reading the panel: accuracy (73%) and AUC (0.77) both clear a reasonable bar at first glance — this is exactly the trap a headline metric sets. The confusion matrix tells a different story: of the restaurants that genuinely don't offer delivery, the model correctly flags only about 1 in 5. It has learned to predict “yes, delivery” almost by default, because most restaurants in this data do offer it. If this model fed a delivery-fleet staffing decision, we'd be dramatically under-planning for the offline-only segment — a real operational miss that accuracy and AUC alone never surface. This is why a single headline number is never the whole evaluation.

Metric	Value	Threshold	Status
Holdout AUC	0.77	≥ 0.70	Within threshold

Holdout accuracy	73%	$\geq 70\%$	Within threshold
Recall, “no delivery” class	~20%	$\geq 50\%$	Flagged

Model 3: Decision Tree — predicting price_range

This is the deliberately broken example. We fit a tree with **no size limit at all** (`cp = 0`, `minsplit = 2`), using predictors that only loosely relate to price — `rating`, `votes`, `platform`, `distance_center_km` — and see what an unconstrained tree does with weak signal.

```
overfit_tree <- rpart(price_range ~ rating + votes + platform + distance_center_km,
                      data = train_data, method = "class",
                      control = rpart.control(cp = 0, minsplit = 2, minbucket = 1))

train_acc_overfit <- mean(predict(overfit_tree, train_data, type = "class") == train_data$price_range)
test_acc_overfit  <- mean(predict(overfit_tree, test_data, type = "class") == test_data$price_range)

cat("Tree size:", sum(overfit_tree$frame$var != "<leaf>"), "splits\n")
```

```
## Tree size: 69 splits
```

```
cat("Training accuracy:", round(train_acc_overfit, 3), "\n")
```

```
## Training accuracy: 1
```

```
cat("Holdout accuracy:", round(test_acc_overfit, 3), "\n")
```

```
## Holdout accuracy: 0.4
```

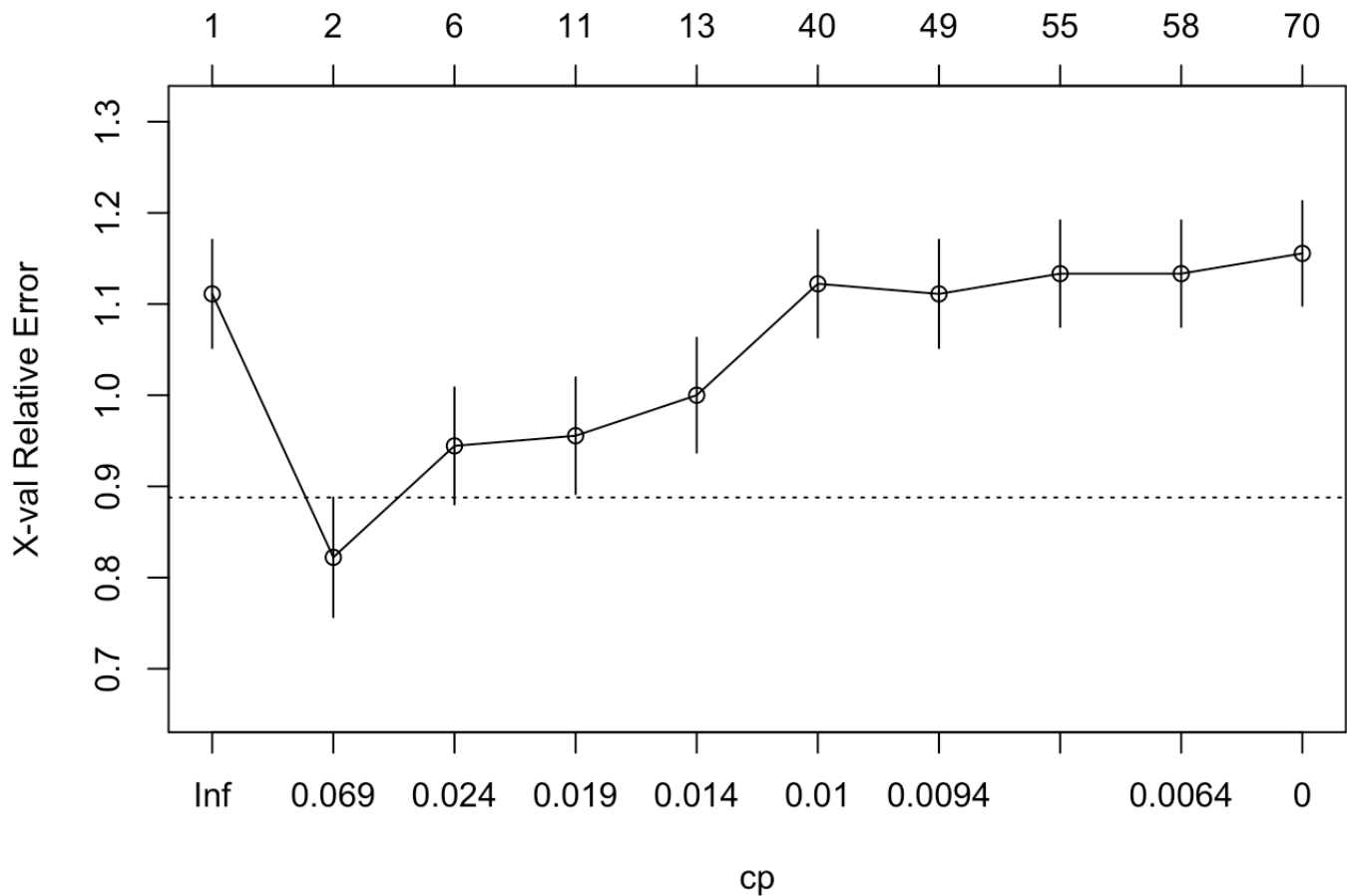
Confirmation re-run: `rpart` already cross-validates internally as it grows the tree, recording a cross-validated error (`xerror`) at every candidate size in `cptable`. This is the same confirm-before-you-trust-it step as the holdout re-run above, done automatically.

```
printcp(overfit_tree)
```

```
##
## Classification tree:
## rpart(formula = price_range ~ rating + votes + platform + distance_center_km,
##       data = train_data, method = "class", control = rpart.control(cp = 0,
##       minsplit = 2, minbucket = 1))
##
## Variables actually used in tree construction:
## [1] distance_center_km platform          rating          votes
##
## Root node error: 90/140 = 0.64286
##
## n= 140
##
##          CP nsplit rel error  xerror    xstd
## 1  0.1888889      0  1.000000  1.11111  0.059391
## 2  0.0250000      1  0.811111  0.82222  0.065627
## 3  0.0222222      5  0.711111  0.94444  0.064207
## 4  0.0166667     10  0.600000  0.95556  0.063994
## 5  0.0111111     12  0.566667  1.00000  0.062994
## 6  0.0098765     39  0.233333  1.12222  0.058937
## 7  0.0088889     48  0.144444  1.11111  0.059391
## 8  0.0074074     54  0.088889  1.13333  0.058464
## 9  0.0055556     57  0.066667  1.13333  0.058464
## 10 0.0000000     69  0.000000  1.15556  0.057459
```

```
plotcp(overfit_tree)
```

size of tree



Reading the panel: the unpruned tree hits 100% training accuracy — it has memorized the training restaurants — but only 40% on the holdout set, worse than guessing among four roughly equal-sized tiers would be by chance. A 60-point gap between training and holdout accuracy is the textbook signature of overfitting: rich enough to fit noise, not general enough to fit the pattern.

Fix — prune to the cross-validated best size:

```
best_cp <- overfit_tree$cptable[which.min(overfit_tree$cptable[, "xerror"]), "CP"]
pruned_tree <- prune(overfit_tree, cp = best_cp)

train_acc_pruned <- mean(predict(pruned_tree, train_data, type = "class") == train_data$price_range)
test_acc_pruned <- mean(predict(pruned_tree, test_data, type = "class") == test_data$price_range)

cat("Pruned tree size:", sum(pruned_tree$frame$var != "<leaf>"), "splits\n")
```

```
## Pruned tree size: 1 splits
```

```
cat("Training accuracy:", round(train_acc_pruned, 3), "\n")
```

```
## Training accuracy: 0.479
```

```
cat("Holdout accuracy:", round(test_acc_pruned, 3), "\n")
```

```
## Holdout accuracy: 0.417
```

Reading the fix: pruning collapses the tree to a single split, and training/holdout accuracy land close together (~48% vs ~42%) instead of 60 points apart. The absolute accuracy is still underwhelming — and that’s the real lesson, not a failure of pruning. Pruning fixed the *overfitting*; it also revealed that `rating`, `votes`, `platform`, and `distance` simply don’t carry much signal about price tier on their own. That’s diagnostics doing its job twice: catching a model that would have shipped with false confidence, and telling the team to go find better features rather than a fancier model.

Metric	Value	Threshold	Status
Unpruned: train – holdout accuracy gap	~60 pts	≤ 15 pts	Flagged
Pruned: train – holdout accuracy gap	~6 pts	≤ 15 pts	Within threshold

Model 4: k-Means Clustering — restaurant segments

```
cluster_features <- scale(restaurant_data[, c("average_cost_two", "rating")])
kmeans_result <- kmeans(cluster_features, centers = 3, nstart = 20)
restaurant_data$cluster <- factor(kmeans_result$cluster)
```

Clustering has no holdout labels to check against — the confirmation re-run here is trying a range of cluster counts and comparing their silhouette scores, the same protocol applied to a different question (“did we pick the right k?” instead of “did we pick the right features?”).

```
silhouette_by_k <- sapply(2:5, function(k) {
  km <- kmeans(cluster_features, centers = k, nstart = 20)
  mean(silhouette(km$cluster, dist(cluster_features))[, 3])
})
names(silhouette_by_k) <- 2:5
print(round(silhouette_by_k, 3))
```

```
##      2      3      4      5
## 0.378 0.422 0.390 0.402
```

```
sil_k3 <- silhouette(kmeans_result$cluster, dist(cluster_features))
cat("Average silhouette, k = 3:", round(mean(sil_k3[, 3]), 3), "\n")
```

```
## Average silhouette, k = 3: 0.422
```

```
ggplot(restaurant_data, aes(x = average_cost_two, y = rating, color = cluster)) +
  geom_point(size = 3, alpha = 0.8) +
  labs(title = "k-Means Clustering (k = 3): Restaurant Segments",
       x = "Average Cost for Two", y = "Rating", color = "Cluster") +
  theme_minimal()
```



Reading the panel: k = 3 is the best of the four values tried, but at 0.42 it sits below our 0.50 bar for “clearly separated” segments — cost and rating alone produce clusters that overlap more than they’re distinct. That’s a useful result to know *before* building a marketing plan around three crisp restaurant personas: the segments are real but fuzzy, so treat cluster boundaries as a starting point for judgment, not a hard rule.

Metric	Value	Threshold	Status
Silhouette, best k (k=3)	0.42	≥ 0.50	Flagged

Second engine: the same evaluate step, as database aggregate functions

Everything above ran fit/score/evaluate in R, in memory. In production, the same evaluate step usually runs as an aggregate query against the platform's database instead of pulling every row into R first. `dplyr`'s `group_by()` + `summarise()` grammar is the same grammar `dbplyr` pushes down to a live SQL database with zero code changes — this is what “database analytical functions to fit, score, and evaluate” looks like in practice: the same protocol, a second engine.

```
restaurant_data$linear_residual <- restaurant_data$estimated_revenue -
  predict(linear_model, newdata = restaurant_data)

restaurant_data %>%
  group_by(platform) %>%
  summarise(
    n_restaurants = n(),
    avg_revenue = round(mean(estimated_revenue), 1),
    avg_abs_error = round(mean(abs(linear_residual)), 1),
    delivery_rate = round(mean(has_online_delivery), 3),
    .groups = "drop"
  )
```

```
## # A tibble: 2 × 5
##   platform n_restaurants avg_revenue avg_abs_error delivery_rate
##   <chr>          <int>         <dbl>         <dbl>         <dbl>
## 1 Swiggy           111         1642.          242.          0.712
## 2 Zomato            89         1776.          277.          0.719
```

```
restaurant_data %>%
  group_by(price_range) %>%
  summarise(
    n_restaurants = n(),
    delivery_rate = round(mean(has_online_delivery), 3),
    avg_rating = round(mean(rating), 2),
    .groups = "drop"
  )
```

```
## # A tibble: 4 × 4
##   price_range n_restaurants delivery_rate avg_rating
##   <fct>          <int>         <dbl>         <dbl>
## 1 Budget           25          0.4          3.08
## 2 Moderate          31         0.516         3.45
## 3 Premium           77         0.753         3.06
## 4 Luxury            67         0.881         3.36
```

Reading the panel: the platform-level breakdown confirms the linear model’s error is stable across Zomato and Swiggy (no platform is quietly dragging the average down), and the price-tier breakdown shows delivery rate climbing steadily from Budget (40%) to Luxury (88%) — consistent with the coefficient logistic regression found, and a sanity check that would take one query to re-run against a live production table with no R involved at all.

Summary: every model, one glance

Model	Metric	Value	Threshold	Status
Linear Regression	Holdout R ²	0.84	≥ 0.60	Within threshold
Linear Regression	Holdout RMSE (% of mean)	~17%	≤ 20%	Within threshold
Logistic Regression	Holdout AUC	0.77	≥ 0.70	Within threshold
Logistic Regression	Recall, “no delivery”	~20%	≥ 50%	Flagged
Decision Tree (unpruned)	Train – holdout gap	~60 pts	≤ 15 pts	Flagged
Decision Tree (pruned)	Train – holdout gap	~6 pts	≤ 15 pts	Within threshold
k-Means (k=3)	Avg. silhouette	0.42	≥ 0.50	Flagged

Three of seven checks are flagged — and that’s the point of running diagnostics on every model, not just the ones that feel shaky. A model can look production-ready on its headline metric and still fail a component check that matters more to the business than the headline does.